DEVS CREATE 313

BUILDING WITH CLAUDE

Idea to First Version

A fill-in-the-blank workbook for tonight's workshop

What you will leave with

- ☐ A tested one-sentence pitch and a wireframe flow you picked
- ☐ A product spec PDF with your build broken into phases
- ☐ A reviewed build plan and the first phase of your project running

How to use this workbook

Each stage has starter prompts. Copy a prompt into Claude and replace anything in **[BRACKETS]** with your own details.

Work inside one project folder all night. What Claude gives you in one stage goes into the next, so save each output (pitch, wireframe, spec PDF) to that folder.

Tonight's pacing

Time	Stage	Minutes
6:00	0. Set up project folder	5
6:05	1. Ideate and validate	15
6:20	2. Wireframes in Claude Design	20
6:40	3. Product spec	20
7:00	4. Build it	45
7:45	Share-out	15

Stage 0: Set up your project folder (5 minutes)

Make a project folder in Claude Code (terminal) or create a Claude Project in the browser if you are on a free plan. Every file you make tonight goes there. In Stage 4, Claude reads your spec from **docs/spec.pdf**, so create a **docs** folder now, or upload the spec to your Project's files if you are in the browser.

STAGE 1 | ABOUT 15 MINUTES

Ideate, or Validate Your Idea

Get to one clear sentence about what you're building and who it's for.

Prompts

STARTER PROMPT A: IF YOU NEED AN IDEA

```
My idea: [YOUR IDEA].

Generate 5 distinct app concepts based on my idea. Write each one in exactly this format:
"An app that [helps/provides a way] for [Persona] who want(s) to [needs/goals/struggles] by
[user story/feature] and [user story/feature]."
```

STARTER PROMPT B: VALIDATE YOUR IDEA (SAVE THE OUTPUT AS YOUR FEEDBACK PDF)

```
You are reviewing an early-stage app idea. Your job is to find the weaknesses before the person spends time building it. Do not praise the idea, do not soften criticism, and do not encourage them to "go for it."

IDEA: [YOUR IDEA, OR THE CONCEPT YOU PICKED FROM PROMPT A]

Work through these in order:

1. Restate the idea in one sentence: who it is for, what problem it solves, and what the app does. If the idea doesn't specify any of these, say which is missing and state the assumption you're making to continue.

2. Problem check. Is this a problem people have and feel, or one the author thinks they should have? What do people do about it today (other apps, spreadsheets, a trainer, nothing)? If the current workaround is "nothing," explain whether that signals an opportunity or a lack of demand.

3. Competition. Name specific existing apps or products that already do this or most of it. Search if you can. For each, state what the idea would do differently. If the difference is a feature a competitor could add in a week, say so.

4. Target user. Identify the narrowest group who would want this most. If the idea targets "everyone," explain why that will fail and propose one specific starting audience.

5. Logic gaps. Quote the exact phrases in the idea that rest on unstated assumptions or vague terms (e.g., "overall health," "ensure," "neglected") and explain what each would need to mean for the app to work.

6. Retention and behavior. Why would someone open this on day 30? What has to be true about user behavior (logging, honesty, consistency) for the core feature to function?

7. Feasibility. Flag anything that is harder to build than it sounds, and any data the app depends on that it may not be able to get.

8. Riskiest assumption. Name the single assumption that, if wrong, kills the idea. Describe the cheapest test to check it in under two weeks, without writing code.

9. Verdict: Build / Rework / Drop, with two or three sentences of reasoning.

Rules:
- Be specific. "The market is crowded" is not acceptable; name the apps.
- If a strength exists, mention it only where it affects the verdict.
- Do not invent statistics. If you're unsure, say so.
- Keep the whole response under 600 words.
```

FOLLOW-UP PROMPT

```
Based on that, rewrite my idea as one sentence in this format: "A [WHAT] for [WHO] so they can [DO WHAT]." Then list the 3 most important things the first version must do, and 3 things I should leave out for now.
```

DEVS CREATE 313

STAGE 2 | ABOUT 20 MINUTES

Generate and Compare Wireframes

Use Claude Design to see a few user flows, then pick one on purpose.

Prompt (in Claude Design)

STARTER PROMPT

```
Create 3 different low-fidelity wireframe user flows for my project so I can compare
layouts.
My project: [ONE-SENTENCE PITCH FROM STAGE 1]
```

FOLLOW-UP PROMPTS

```
Compare the 3 flows for a first-time user. Which one is easiest to understand in 5 seconds,
and why?
```

```
Take Option [LETTER] and change [WHAT YOU WANT DIFFERENT]. Keep everything else the same.
```

Save tokens: edit one flow until you like it rather than regenerating all three. Export the one you pick as your wireframe PDF and save it in your project folder.

STAGE 3 | ABOUT 20 MINUTES

Write Your Product Spec

Claude interviews you, then writes a spec PDF your build agent will follow.

Prompt

STARTER PROMPT (ATTACH YOUR WIREFRAME)

```
I have an idea for a digital product: [ONE-SENTENCE PITCH FROM STAGE 1].

Help me turn this into a PDF product spec I can later hand to AI coding agents. Break work
into phases or sprints. Have the AI build one phase carefully, then commit before moving on.

Step 1 - Interview me with the AskUserQuestion tool.
- Ask in two rounds of 3-4 multiple-choice questions each.
- Round 1: the big product decisions (platform, how users input data, how goals/targets
work, business model).
- Round 2: based on my Round 1 answers, ask about my coding experience, level of
detail/scope for v1, and the most important user interaction.
- Mark a recommended option when there's a sensible default.
- Don't write anything until both rounds are answered.

Step 2 - Write a spec doc in PDF format (not a chat reply) with these sections:
1. App idea - one-paragraph pitch, the problem it solves, and the core user loop.
2. Product decisions - a table: Question | My decision | What it means for the build.
3. Functional spec - key screens and features, the core logic/rules, a simple data model,
and a free vs. paid feature table if relevant.
4. Recommended build phases - 6-8 small phases, each ending in something I can run. For each
phase write an "Agent task" paragraph I can paste to an AI agent, plus a checklist of
acceptance criteria. Include a simple roadmap diagram of the phases.
5. Working with agents - how to use this spec with agents, constraints for every phase, and
what's out of scope for v1.

Keep the tech stack beginner-friendly if I say I'm new to coding, and stick to the decisions
I made in the interview.
```

Save the result as **docs/spec.pdf** in your project folder. Stage 4 reads it from there.

STAGE 4 | ABOUT 45 MINUTES

Build It

Claude reviews your spec, plans the phases, and builds one phase at a time.

How this works

Claude stops after the plan and after every phase. Read what it reports, test it, then reply **approved** or send changes. When something breaks, copy the exact error message into Claude. Don't describe it from memory.

STARTER PROMPT (FIRST MESSAGE IN YOUR PROJECT)

```
You are building the app described in docs/spec.pdf. The spec is the source of truth. Work
in phases and stop for my approval at each gate.

PHASE 0 - SPEC REVIEW (no code)
Read the entire spec before writing anything. Then produce docs/plan.md containing:
1. A one-paragraph summary of what the app does and who uses it.
2. Every feature and requirement in the spec, each with an ID (R1, R2...) and the page
number it came from.
3. Gaps and contradictions: anything the spec leaves undefined, contradicts itself on, or
that you would otherwise have to guess. For each, give the question and your proposed
default.
4. A phased build plan. Each phase must:
   - deliver something I can run and test on its own
   - list the requirement IDs it covers
   - list acceptance criteria as testable statements ("User can X and sees Y")
   - name the files/modules it creates or changes
   Order phases so data models and core flows come before polish, and so no phase depends on
a later one.
5. Tech stack and architecture decisions, each with a one-line reason tied to the spec.
Stop after writing plan.md. Do not start Phase 1 until I reply "approved" or send changes.

FOR EACH BUILD PHASE
- Build only what that phase lists. If you need something from a later phase, stub it and
note it.
- Do not add features, screens, or dependencies the spec doesn't call for. If you think one
is needed, ask.
- If you hit an ambiguity not already in the gaps list, add it to plan.md and use the most
conservative option.
- Before reporting done: build the project, run tests, and check each acceptance criterion.
Mark each pass/fail with how you verified it.
- End the phase with a short report: what was built, criteria results, deviations from the
spec, open questions, and what the next phase will do.
- Stop and wait for "approved" before the next phase.

Keep plan.md updated as the running record: mark phases complete, log every decision I make
in answer to your questions.
```

WHEN SOMETHING BREAKS

```
Something went wrong in Phase [NUMBER]. I was trying to [WHAT YOU DID].
I expected [WHAT SHOULD HAVE HAPPENED], but instead [WHAT HAPPENED].
Here's the exact error message: [PASTE ERROR]
Explain what went wrong in plain words, then give me the fix.
```

WHEN YOU'RE LOST

```
I'm confused. Explain what we've built so far in 3 sentences, what the code I just added
does, and what we're doing next.
```

NEXT STEPS

Keep Going

PROMPT TO KEEP GOING AT HOME

```
Read docs/spec.pdf and docs/plan.md. Phase [NUMBER] is approved. Start Phase [NEXT NUMBER]
using the same rules: build only what it lists, check each acceptance criterion, report, and
wait for "approved."
```

Tips that apply to every stage

- ☐ Be specific. "A sign-up sheet for my church's food pantry volunteers" gets a better answer than "a volunteer app."
- ☐ Ask for one thing at a time. Big requests produce big, hard-to-fix results.
- ☐ When an answer is confusing, say so and ask Claude to explain it more simply.
- ☐ Paste exact error messages and screenshots instead of describing them.
- ☐ Claude can be wrong. If something doesn't work, it isn't your fault. Report what happened and try the fix.